



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра информационных технологий в атомной энергетике

ОТЧЕТ ПО ПРАКТИКЕ №6

«Автоматизированная система управления для обеспечения инвентарем для лыжного спорта»

по дисциплине «Автоматизированные системы управления элементами
промышленных и административных объектов»

Студент группы ИКБО-50-23

Павлов Н.С.

(подпись студента)

Принял преподаватель

Щербаков К. В.

(подпись руководителя)

Работа представлена

« ____ » _____ 2026 г.

Допущен к работе

« ____ » _____ 2026 г.

Москва 2026

ОГЛАВЛЕНИЕ

1. Анализ функций АСУ и определение функции для доработки	3
2. Анализ существующих языков программирования и IDE	5
3. Обоснованный выбор языка, IDE и библиотек	6
4. Генерация программного кода.....	8

1. Анализ функций АСУ и определение функции для доработки

Выбранная функция для разработки: Функция 1.7 — «Формирование плана закупок на период» (из Модуля 1: Управление закупками, планирование и бюджетный контроль).

Внутрисистемные функции, реализуемые модулем:

- 1.IS19 — Запуск алгоритма расчета потребности (анализ утвержденных заявок)
- 1.IS20 — Анализ текущих остатков на складе по каждой позиции
- 1.IS21 — Учет минимальных страховых запасов при расчете потребности
- 1.IS22 — Генерация проекта плана закупок с группировкой по номенклатуре

Обоснование выбора:

1. Высокая алгоритмическая сложность — прогнозирование требует применения методов машинного обучения (временные ряды, сезонность).

2. Специфичность предметной области — лыжный спорт имеет выраженную сезонность:

- Подготовка к зимнему сезону (август-октябрь)
- Пик использования (ноябрь-март)
- Межсезонное обслуживание (апрель-июль)

3. Учет множества факторов:

- Исторические данные по расходу инвентаря
- Сезонные коэффициенты спроса
- Нормативный износ лыж (зависит от интенсивности использования)
- Планируемые соревнования и сборы
- Рост/смена состава спортсменов (изменение размеров)

4. Экономическая значимость — ошибки прогнозирования приводят к:

- Дефициту (срыв тренировочного процесса)
- Избытку (заморозка бюджета, устаревание моделей)

5. Отсутствие готовых решений — типовые ERP-системы не учитывают специфику износа лыжного инвентаря.

2. Анализ существующих языков программирования и IDE

Таблица 2.1 – Рассмотренные языки программирования для задачи прогнозирования

Язык	Преимущества	Недостатки	Применимость к задаче
Python	Богатейшая экосистема для ML (pandas, scikit-learn, Prophet, statsmodels); быстрая разработка; интерактивный анализ в Jupyter	Динамическая типизация; ниже производительность на больших данных; GIL ограничивает многопоточность	Высокая — идеален для прототипирования и реализации ML-моделей прогнозирования
R	Специализирован для статистического анализа; отличная визуализация (ggplot2); множество пакетов для временных рядов	Сложный синтаксис; меньшее сообщество в РФ; сложнее интеграция с Java-бэкендом	Средняя — хорош для исследовательского анализа, но сложен в production
Java	Высокая производительность; строгая типизация; отличная интеграция с существующей архитектурой (Spring Boot)	Ограниченная поддержка ML-библиотек (Weka, Smile уступают Python); громоздкий код	Средняя — хорош для production-контура, но не для разработки моделей
Julia	Высокая производительность; синтаксис, близкий к математическому	Молодая экосистема; мало библиотек; сложность развертывания	Низкая — неоправданные риски для production

Таблица 2.2 – Рассмотренные IDE для разработки ML-модуля

IDE	Поддерживаемые языки	Преимущества	Недостатки
PyCharm Professional	Python, SQL, JavaScript	Лучшая поддержка научных библиотек; встроенный Jupyter Notebook; отладка DataFrame; интеграция с БД	Платная (бесплатна для студентов)
JupyterLab	Python, R, Julia	Интерактивная разработка; визуализация «на лету»; идеален для экспериментов	Не подходит для полноценного production-кода; нет рефакторинга
VS Code	Python, R, SQL	Бесплатный; отличные расширения для Python; поддержка Jupyter внутри редактора	Требует настройки окружения
DataSpell	Python, R, SQL	Специализированная IDE для Data Science от JetBrains; встроенный Jupyter	Платная; избыточна для небольшого модуля

3. Обоснованный выбор языка, IDE и библиотек

Выбранный язык программирования: Python 3.12

Обоснование:

- Библиотеки для временных рядов — Prophet (Facebook/Meta), statsmodels, scikit-learn — готовые реализации алгоритмов прогнозирования.
- Быстрое прототипирование — возможность проверить гипотезы в Jupyter Notebook перед интеграцией.
- Интеграция с Java-бэкендом — через REST API или экспорт модели в PMML/ONNX.
- Визуализация — matplotlib, seaborn, plotly для построения графиков прогнозов.
- Работа с данными — pandas для ETL-операций, загрузки из PostgreSQL.
- Соответствие архитектуре ПП5 — Python указан как язык аналитической подсистемы (позиция 2.4-2.5).

Выбранная IDE: PyCharm Professional 2026.1

Обоснование:

- Бесплатная лицензия для студентов — доступ к Professional версии.
- Встроенная поддержка Jupyter Notebook — можно вести разработку модели в .ipynb и затем экспортировать в .py.
- Интеграция с БД — просмотр таблиц PostgreSQL, выполнение SQL-запросов без переключения.
- Отладка DataFrame — просмотр содержимого pandas DataFrame в удобном табличном виде.
- Поддержка научных библиотек — автодополнение для pandas, numpy, sklearn.

Таблица 3.1 – Выбранные библиотеки Python

Библиотека	Версия	Назначение	Обоснование
pandas	2.2.0	Обработка и анализ данных	Стандарт для работы с табличными данными, загрузка из SQL
numpy	1.26.0	Численные вычисления	Базовые математические операции, работа с массивами
statsmodels	0.14.1	Статистический анализ	Экспоненциальное сглаживание (Holt-Winters), тесты на стационарность
scikit-learn	1.4.0	Машинное обучение	Метрики качества прогноза (MAE, MAPE, RMSE)
psycopg2	2.9.9	Подключение к PostgreSQL	Загрузка исторических данных из БД АСУ
sqlalchemy	2.0.25	ORM для Python	Удобная работа с БД через pandas
loguru	0.7.2	Логирование	Удобное логирование процесса прогнозирования
joblib	1.3.2	Сериализация моделей	Сохранение обученной модели для повторного использования
python-dotenv	1.0.0	Переменные окружения	Загрузка конфигурации из файла .env

4. Генерация программного кода

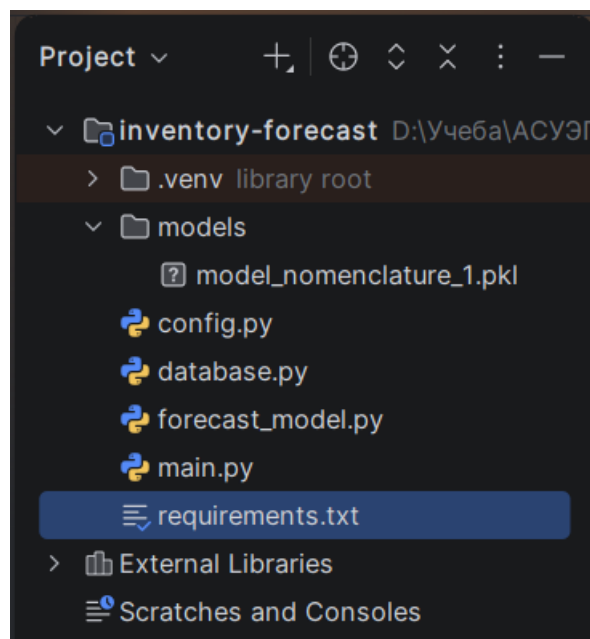


Рисунок 4.1 – Структура разрабатываемого модуля

Листинг 1 - Файл requirements.txt

```
pandas==2.2.0
numpy==1.26.0
statsmodels==0.14.1
scikit-learn==1.4.0
psycopg2-binary==2.9.9
sqlalchemy==2.0.25
matplotlib==3.8.2
loguru==0.7.2
joblib==1.3.2
python-dotenv==1.0.0
```

Листинг 2 – Файл config.py

```
"""Конфигурация модуля прогнозирования."""
import os
from dataclasses import dataclass, field

@dataclass
class DatabaseConfig:
    """Настройки подключения к PostgreSQL."""
    host: str = field(default_factory=lambda: os.getenv("DB_HOST", "localhost"))
    port: int = field(default_factory=lambda: int(os.getenv("DB_PORT", "5432")))
    database: str = field(default_factory=lambda: os.getenv("DB_NAME", "postgres"))
    schema: str = field(default_factory=lambda: os.getenv("DB_SCHEMA", "asu_inventory"))
    username: str = field(default_factory=lambda: os.getenv("DB_USERNAME", "postgres"))
    password: str = field(default_factory=lambda: os.getenv("DB_PASSWORD", ""))
```



```

    @property
    def connection_string(self) -> str:
        return f"postgresql://{self.username}:{self.password}@{self.host}:{self.port}/{self.database}"

@dataclass
class ForecastConfig:
    """Настройки модели прогнозирования."""
    forecast_days: int = 90
    seasonality_mode: str = "multiplicative"
    yearly_seasonality: bool = True
    weekly_seasonality: bool = True
    min_train_days: int = 30

@dataclass
class AppConfig:
    """Общая конфигурация приложения."""
    db: DatabaseConfig = field(default_factory=DatabaseConfig)
    forecast: ForecastConfig = field(default_factory=ForecastConfig)
    models_dir: str = field(default_factory=lambda: os.getenv("MODELS_DIR",
"./models"))

config = AppConfig()

```

Листинг 3 – Файл database.py

```

"""Модуль для работы с базой данных PostgreSQL."""
import pandas as pd
from sqlalchemy import create_engine, text
from loguru import logger
from typing import Optional, Dict, Any

from config import config

class DatabaseManager:
    """Менеджер для работы с БД АСУ."""

    def __init__(self):
        self.engine = create_engine(
            config.db.connection_string,
            connect_args={'options': f'-c search_path={config.db.schema},public'}
        )
        logger.info(f"Подключение к БД: {config.db.host}:{config.db.port}/{config.db.database}")
        logger.info(f"Схема: {config.db.schema}")

    def get_historical_consumption(self, nomenclature_id: int, start_date: str, end_date: Optional[str] = None) -> pd.DataFrame:
        """Загрузка исторических данных по расходу инвентаря."""
        query = f"""
        SELECT
            DATE(d.issuance_date) as ds,
            COUNT(*) as y
        FROM issuance_document d
        JOIN inventory_item i ON d.inventory_item_id = i.inventory_item_id
        WHERE i.nomenclature_id = :nomenclature_id
            AND d.issuance_date >= :start_date
        """

```

```

        """
        params = {"nomenclature_id": nomenclature_id, "start_date":
start_date}

        if end_date:
            query += " AND d.issuance_date <= :end_date"
            params["end_date"] = end_date

        query += " GROUP BY DATE(d.issuance_date) ORDER BY ds"

        logger.info(f"Загрузка данных для номенклатуры ID={nomencla-
ture_id}")

        with self.engine.connect() as conn:
            df = pd.read_sql(text(query), conn, params=params)

            if not df.empty:
                df['ds'] = pd.to_datetime(df['ds'])
                date_range = pd.date_range(start=df['ds'].min(),
end=df['ds'].max())
                df = df.set_index('ds').reindex(date_range, fill_value=0).re-
set_index()
                df.columns = ['ds', 'y']

            logger.info(f"Загружено {len(df)} записей")
            return df

    def get_current_stock(self, nomenclature_id: int) -> int:
        """Получение текущего остатка на складе."""
        query = f"""
        SELECT COUNT(*) as stock
        FROM inventory_item
        WHERE nomenclature_id = :nomenclature_id
            AND inventory_status = 'AVAILABLE'
        """
        with self.engine.connect() as conn:
            result = conn.execute(text(query), {"nomenclature_id": nomencla-
ture_id})
            return result.scalar() or 0

    def get_nomenclature_info(self, nomenclature_id: int) -> Dict[str, Any]:
        """Получение информации о номенклатуре."""
        query = f"""
        SELECT
            nomenclature_id,
            nomenclature_name,
            nomenclature_category,
            min_stock_level,
            standard_service_life,
            manufacturer
        FROM nomenclature
        WHERE nomenclature_id = :nomenclature_id
        """
        with self.engine.connect() as conn:
            result = conn.execute(text(query), {"nomenclature_id": nomencla-
ture_id})
            row = result.fetchone()
            if row:
                return dict(row._mapping)
            return {}

    def get_all_nomenclature(self) -> pd.DataFrame:
        """Получение списка всей номенклатуры."""
        query = f"""

```

```

SELECT
    nomenclature_id,
    nomenclature_name,
    nomenclature_category,
    min_stock_level,
    manufacturer
FROM nomenclature
ORDER BY nomenclature_id
"""

with self.engine.connect() as conn:
    return pd.read_sql(text(query), conn)

```

Листинг 4 – Файл *forecast model.py*

```

"""Модель прогнозирования потребности в лыжном инвентаре (statsmodels)."""
import pandas as pd
import numpy as np
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from loguru import logger
import joblib
import os
from typing import Optional, Dict, Any
from datetime import datetime, timedelta

from config import config

class SkiInventoryForecastModel:
    """Модель прогнозирования на основе экспоненциального сглаживания."""

    def __init__(self, model_name: str = "ski_forecast"):
        self.model_name = model_name
        self.model = None
        self.training_metrics: Dict[str, float] = {}
        self._simple_mean = None
        self._simple_std = None
        self._seasonal_periods = 7

    def prepare_data(self, df: pd.DataFrame):
        """Подготовка данных."""
        df = df.copy()
        df['ds'] = pd.to_datetime(df['ds'])
        df = df.set_index('ds')
        df = df.asfreq('D', fill_value=0)
        series = df['y'].astype(float)
        logger.info(f"Данные подготовлены: {len(series)} точек")
        return series

    def fit(self, df: pd.DataFrame) -> 'SkiInventoryForecastModel':
        """Обучение модели."""
        logger.info(f"Обучение модели {self.model_name}...")
        series = self.prepare_data(df)

        if len(series) < config.forecast.min_train_days:
            logger.warning(f"Мало данных: {len(series)} точек. Использую среднее.")
            self._simple_mean = series.mean()
            self._simple_std = series.std()
            return self

        # Определяем seasonal_periods на основе длины ряда
        data_len = len(series)
        if data_len < 14:

```

```

        self._seasonal_periods = 3
        use_seasonal = False
        use_trend = False
    elif data_len < 30:
        self._seasonal_periods = 5
        use_seasonal = True
        use_trend = False
    elif data_len < 60:
        self._seasonal_periods = 7
        use_seasonal = True
        use_trend = False
    else:
        self._seasonal_periods = 7
        use_seasonal = True
        use_trend = True

    logger.info(f"Параметры: seasonal_periods={self._seasonal_periods},
"
               f"seasonal={use_seasonal}, trend={use_trend}")

    try:
        if use_seasonal:
            self.model = ExponentialSmoothing(
                series,
                seasonal_periods=self._seasonal_periods,
                trend='add' if use_trend else None,
                seasonal='add',
                damped_trend=True,
                initialization_method='estimated'
            ).fit()
        else:
            self.model = ExponentialSmoothing(
                series,
                trend=None,
                seasonal=None,
                initialization_method='estimated'
            ).fit()

        fitted = self.model.fittedvalues
        valid_idx = ~np.isnan(fitted)
        if valid_idx.sum() > 0:
            mae = np.mean(np.abs(series[valid_idx] - fitted[valid_idx]))
            mape = np.mean(np.abs((series[valid_idx] - fitted[valid_idx]) /
                                np.where(series[valid_idx] == 0, 1,
series[valid_idx]))) * 100
            self.training_metrics = {'mae': round(mae, 2), 'mape':
round(mape, 1)}
            logger.info(f"Модель обучена. MAE={mae:.2f},
MAPE={mape:.1f}%")
        except Exception as e:
            logger.error(f"Ошибка обучения: {e}. Использую среднее.")
            self.model = None
            self._simple_mean = series.mean()
            self._simple_std = series.std()

    return self

def predict(self, periods: int = None) -> pd.DataFrame:
    """Прогноз."""
    if periods is None:
        periods = config.forecast.forecast_days

    start_date = datetime.now().date()

```

```

        dates = [start_date + timedelta(days=i) for i in range(periods)]

        if self.model is not None:
            try:
                forecast_values = self.model.forecast(steps=periods)
                forecast_values = np.maximum(forecast_values, 0)

                # Ограничиваем рост: не больше максимума из исторических
                max_historical = self._simple_mean * 3 if self._simple_mean
            else 5
                forecast_values = np.minimum(forecast_values, max_historical)

            return pd.DataFrame({
                'ds': dates,
                'yhat': forecast_values,
                'yhat_lower': np.maximum(forecast_values * 0.6, 0),
                'yhat_upper': np.minimum(forecast_values * 1.5, max_historical * 1.5)
            })
        except Exception as e:
            logger.error(f"Ошибка прогноза: {e}")

        # Запасной вариант: простое среднее
        mean_val = self._simple_mean or 0
        std_val = self._simple_std or 0
        values = np.full(periods, mean_val)
        return pd.DataFrame({
            'ds': dates,
            'yhat': values,
            'yhat_lower': np.maximum(values - 2 * std_val, 0),
            'yhat_upper': values + 2 * std_val
        })

    def save(self, path: Optional[str] = None):
        if path is None:
            path = os.path.join(config.models_dir, f"{self.model_name}.pkl")
            os.makedirs(os.path.dirname(path), exist_ok=True)
            joblib.dump({
                'model': self.model,
                'model_name': self.model_name,
                'training_metrics': self.training_metrics,
                '_simple_mean': self._simple_mean,
                '_simple_std': self._simple_std,
                '_seasonal_periods': self._seasonal_periods
            }, path)
            logger.info(f"Модель сохранена: {path}")

    @classmethod
    def load(cls, path: str) -> 'SkiInventoryForecastModel':
        data = joblib.load(path)
        instance = cls(model_name=data['model_name'])
        instance.model = data['model']
        instance.training_metrics = data['training_metrics']
        instance._simple_mean = data.get('_simple_mean')
        instance._simple_std = data.get('_simple_std')
        instance._seasonal_periods = data.get('_seasonal_periods', 7)
        logger.info(f"Модель загружена: {path}")
        return instance

```

Листинг 5 – Файл main.py

```

"""Главный скрипт запуска модуля прогнозирования."""
import os
from datetime import datetime
from loguru import logger

from config import config
from database import DatabaseManager
from forecast_model import SkiInventoryForecastModel

def print_header(title: str):
    """Вывод заголовка раздела."""
    print("\n" + "=" * 70)
    print(f" {title}")
    print("=" * 70)

def main():
    """Основная функция."""
    print_header("МОДУЛЬ ПРОГНОЗИРОВАНИЯ ПОТРЕБНОСТИ В ЛЫЖНОМ ИНВЕНТАРЕ")

    # 1. Подключение к БД
    print_header("1. ПОДКЛЮЧЕНИЕ К БАЗЕ ДАННЫХ")
    db = DatabaseManager()

    # 2. Загрузка списка номенклатуры
    print_header("2. ДОСТУПНАЯ НОМЕНКЛАТУРА")
    nomenclature_df = db.get_all_nomenclature()
    print(nomenclature_df.to_string(index=False))

    # 3. Выбор номенклатуры для прогноза
    print_header("3. ВЫБОР НОМЕНКЛАТУРЫ")
    nomenclature_id = 1
    info = db.get_nomenclature_info(nomenclature_id)
    print(f"Выбрана номенклатура: ID={nomenclature_id}")
    print(f" Наименование: {info.get('nomenclature_name', '-')}")
    print(f" Категория: {info.get('nomenclature_category', '-')}")
    print(f" Производитель: {info.get('manufacturer', '-')}")
    print(f" Мин. остаток: {info.get('min_stock_level', '-')} шт.")
    print(f" Срок службы: {info.get('standard_service_life', '-')} мес.")

    # 4. Загрузка исторических данных
    print_header("4. ЗАГРУЗКА ИСТОРИЧЕСКИХ ДАННЫХ")
    start_date = '2024-01-01'
    df = db.get_historical_consumption(nomenclature_id, start_date)
    print(f"Загружено {len(df)} записей (с {df['ds'].min().date() if not df.empty else '-'} "
          f"по {df['ds'].max().date() if not df.empty else '-'}")

    if len(df) > 0:
        print("\nПервые 5 записей:")
        print(df.head().to_string(index=False))
        print(f"\nСтатистика: среднее={df['y'].mean():.1f} шт./день, "
              f"максимум={df['y'].max():.0f} шт./день")

    # 5. Обучение модели
    print_header("5. ОБУЧЕНИЕ МОДЕЛИ")
    model = SkiInventoryForecastModel(model_name=f"forecast_nomenclature_{nomenclature_id}")
    model.fit(df)
    print(f"Метрики качества:")
    if model.training_metrics:
        print(f" MAE (средняя абсолютная ошибка):

```

```

{model.training_metrics['mae']} шт.")
    print(f"    MAPE (средняя процентная ошибка): {model.training_metrics['mape']}%")

# 6. Прогноз
print_header("6. ПРОГНОЗ НА 90 ДНЕЙ")
forecast = model.predict(periods=90)

forecast['month'] = forecast['ds'].apply(lambda x: x.month)
month_names = {5: 'Май', 6: 'Июнь', 7: 'Июль', 8: 'Август', 9: 'Сентябрь', 10: 'Октябрь'}
forecast['month_name'] = forecast['month'].map(month_names)
monthly = forecast.groupby('month_name')['yhat'].sum().round(1)

print("\nПрогноз потребности по месяцам:")
for month, value in monthly.items():
    bar = "█" * max(1, int(value))
    print(f"    {month:10}: {value:6.1f} шт.    {bar}")

total_demand = forecast['yhat'].sum()
total_lower = forecast['yhat_lower'].sum()
total_upper = forecast['yhat_upper'].sum()

print(f"\nСуммарный прогноз на 90 дней:")
print(f"    Ожидаемый спрос:        {total_demand:.1f} шт.")
print(f"    Доверительный интервал: [{total_lower:.1f} - {total_upper:.1f}] шт.")

# 7. Сравнение с текущим остатком
print_header("7. РЕКОМЕНДАЦИИ ПО ЗАКУПКЕ")
current_stock = db.get_current_stock(nomenclature_id)
min_stock = info.get('min_stock_level', 5)

print(f"    Текущий остаток:        {current_stock} шт.")
print(f"    Минимальный запас:      {min_stock} шт.")
print(f"    Прогноз спроса:         {total_demand:.1f} шт.")

if current_stock >= total_demand + min_stock:
    print(f"\n    ✓ Запас достаточен. Закупка не требуется.")
else:
    need = total_demand + min_stock - current_stock
    print(f"\n    ! Рекомендуется закупить: {need:.1f} шт.")

# 8. Сохранение модели
print_header("8. СОХРАНЕНИЕ МОДЕЛИ")
model_path = os.path.join(config.models_dir, f"model_nomenclature_{nomenclature_id}.pkl")
model.save(model_path)
print(f"    Модель сохранена в: {model_path}")

print_header("РАБОТА МОДУЛЯ ЗАВЕРШЕНА")

if __name__ == "__main__":
    main()

```